

Project 3: 6DOF Pose Estimation for Warehouse Object Picking

Using Classical Depth and RGB Feature Fusion

ENGG*6100 Machine Vision

Antony Gerold Arockiasamy - 1404714

March 27, 2026

Abstract

I built two classical pipelines for 6DOF pose estimation on the Rutgers APC RGB-D dataset (24 objects, 432 images each). V1 uses depth-only methods (ICP, render-and-compare, flip correction) and achieves acceptable accuracy ($T < 25\text{mm}$, $R < 15^\circ$) on 17/24 objects. Analysing V1's failures showed that the bottleneck was unreliable translation from depth seed finding. Testing PnP separately revealed it gives excellent translation (median 3mm) but random rotation (median 166°). Fusing PnP translation with depth rotation gives V2, which passes 21/24 (88%). The math: V1 passes 17, kyjen regresses (-1) , 5 objects rescued $(+5)$, giving 21. The key insight is that RGB finds where the object is, depth finds how it is oriented, and neither alone is enough.

1 Introduction

6DOF pose estimation recovers the 3D position and orientation of a known object from sensor data. This is essential for robotic warehouse picking, where a robot arm needs to know exactly where an object is and how it is rotated before grasping. The Amazon Picking Challenge [2] brought together multiple teams to solve this for household objects in shelf bins, and the Rutgers APC RGB-D dataset [1] was created to support this research.

My goal was to understand how far classical methods can take us, and what each modality contributes. I avoided deep learning to build intuition for the geometric foundations: ICP for point cloud registration [4], solvePnP for camera geometry [5], and render-and-compare for template matching.

1.1 Dataset

The Rutgers APC dataset [1] has 24 household objects across 12 shelf bins $\times$ 36 viewpoints = 432 images per object. Each image includes Kinect v1 depth (uint16 mm), RGB, a binary bin mask, and ground truth 6DOF pose. 3D models are STL (geometry) and OBJ with PNG textures. Camera: $f_x=572.41$, $f_y=573.57$, $c_x=325.26$, $c_y=242.05$.

I trace everything through five objects (Figure 1): expo eraser (both pass), cheezit (both pass, texture-rich), sharpie (V1 fails, V2 fixes), dr_browns (V1 fails, V2 fixes), and kyjen eggs (V1 passes, V2 regresses).

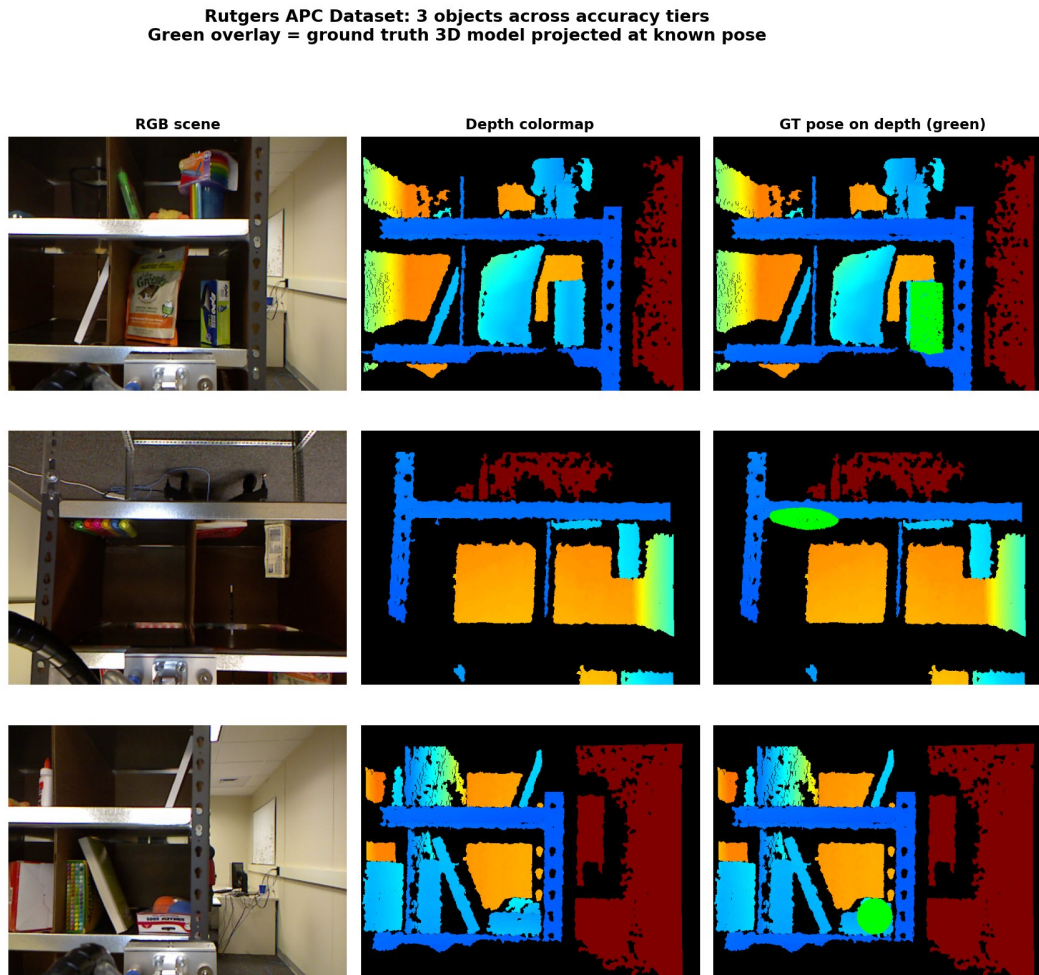


Figure 1: Five objects used throughout this report. Left: RGB scene. Centre: depth colormap. Right: GT 3D model projected in green. Top two rows are Excellent objects, middle two are V2 rescue cases, bottom row regresses under V2.

1.2 Evaluation

Three tiers: Excellent ($T < 10\text{mm}$, $R < 5^\circ$), Acceptable ($T < 25\text{mm}$, $R < 15^\circ$, the main threshold), and Mismatch. Both pipelines use a depth prior of $\pm 50\text{mm}$ around the shelf depth for initial localisation, mimicking real shelf calibration [2]. No ground truth rotation or X/Y translation is used.

2 Environment Analysis

Before building any pipeline, I analysed the data to understand the challenges.

2.1 The Bin Mask Problem

The masks outline the shelf compartment, not the object. Within one mask, three depth layers compete: shelf edge ($\sim 520\text{mm}$), object ($\sim 600\text{mm}$), and back wall ($\sim 930\text{mm}$). Figure 2 shows the histogram with three clear peaks.

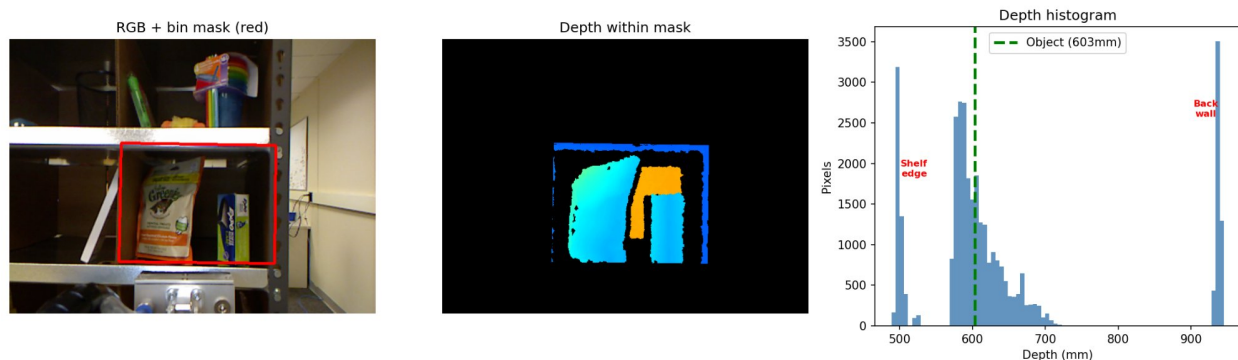


Figure 2: Left: mask covers the compartment (red outline). Centre: depth within mask. Right: histogram with three peaks. Green line marks object depth (603mm). Filtering by this depth isolates the object.

I tried five strategies to isolate the object without the depth prior: P25 percentile, histogram peak, floor subtraction, largest connected component, and centre blob. None matched simple depth filtering ($\pm 50\text{mm}$). Understanding this environment was the first critical step.

2.2 PnP Exploration

Before committing to a depth-only approach, I tested whether RGB could solve the problem directly. I extracted ORB features [6] from the RGB image, obtained 3D coordinates from the depth image at those locations, and ran solvePnP Ransac [5].

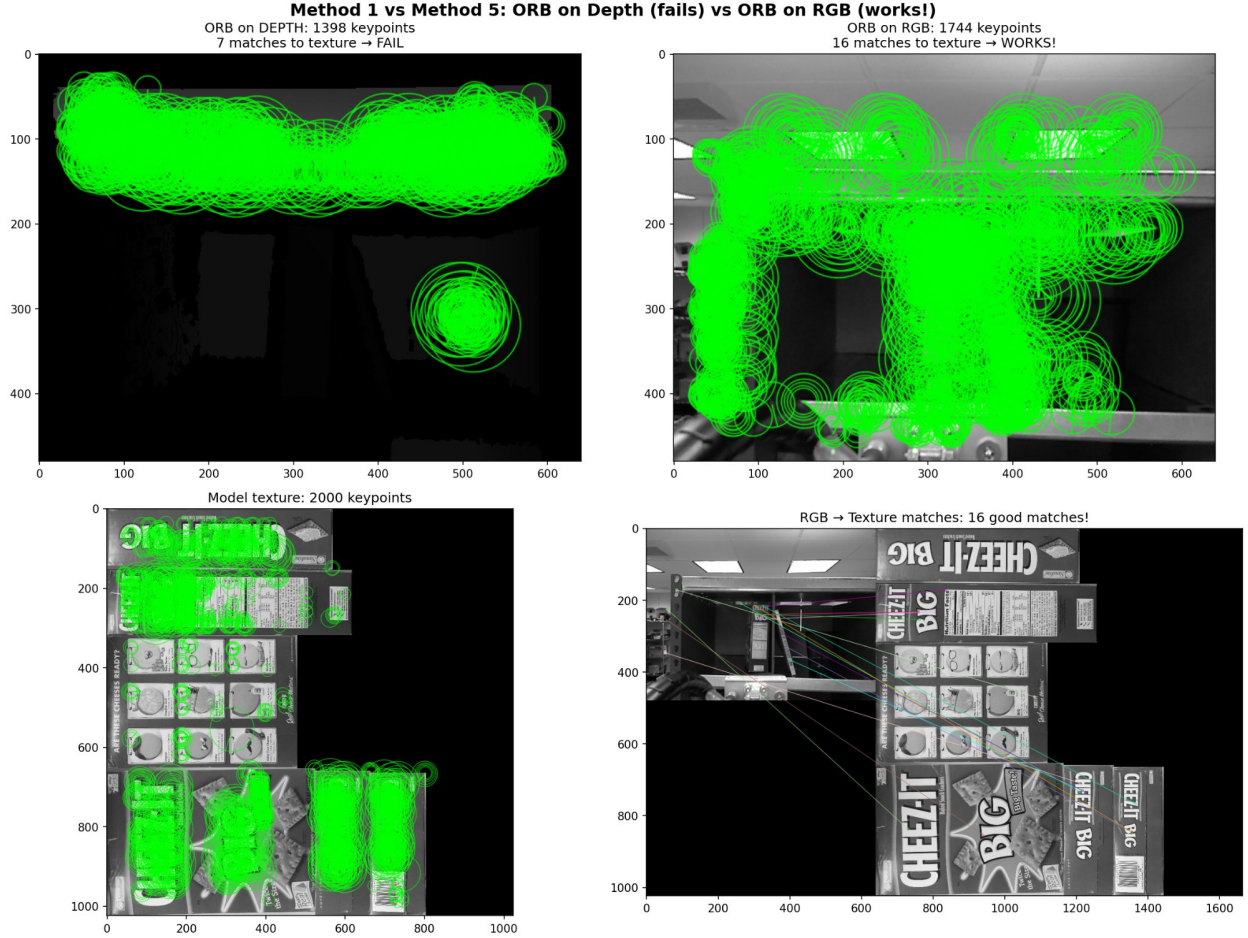


Figure 3: ORB on depth (left, 1398 keypoints, 7 matches, fails) vs ORB on RGB (right, 1744 keypoints, 16 matches, works). Bottom: model texture keypoints and RGB-to-texture correspondences.

The result was surprising: median $T=3\text{mm}$ (excellent) but $R=166^\circ$ (random). Figure 3 shows why. ORB on the RGB image produces discriminative texture features that match the model, giving precise translation. But with only 16 sparse correspondences clustered on one face of the object, the EPnP solver [5] cannot constrain the rotation. PnP finds *where* an object is but not *how it is oriented*.

This told me two things: (1) I need depth for rotation, since depth silhouettes encode 3D shape directly, and (2) if depth-based translation ever fails, PnP can provide it. I built V1 (depth only) first, then used PnP to fix V1’s translation failures in V2.

3 V1: Depth-Geometric Pipeline

Four stages, each fixing a failure of the previous one.

Stage 1: Seed Finding. Filter mask pixels by depth prior ($\pm 50\text{mm}$), select 40 seeds, back-project to 3D using $\mathbf{p}_{3D} = [(u - c_x)Z/f_x, (v - c_y)Z/f_y, Z]$. When seeds land on

the object, subsequent stages succeed. When they land on shelf structure at similar depth, everything fails. This is V1’s bottleneck.

Stage 2: ICP Alignment. For each seed, extract a local scene cloud (60px radius), try 16 initial rotations, run 25 iterations of ICP [4] with the model downsampled to ~ 400 points. Select the best coverage pair. ICP refines translation but rotation depends heavily on the initial guess.

Stage 3: Render-and-Compare. Keep ICP translation, search ~ 200 rotations by rendering model depth silhouettes:

$$S_{\text{render}} = \frac{|\{p : |D_{\text{render}}(p) - D_{\text{obs}}(p)| < 5\text{mm}\}|}{|\{p : D_{\text{render}}(p) > 0 \wedge D_{\text{obs}}(p) > 0\}|} \quad (1)$$

Fixes rotation for asymmetric objects but creates 180° ambiguity for symmetric silhouettes.

Stage 4: Flip Correction. Test 180° rotations around X/Y/Z, re-score, keep best.

3.1 How Stages Improve the Pose

Figure 4 shows the champion spark plug through all four stages. The estimated model (red) starts far from the scene (teal) after seed+ICP ($T=130\text{mm}$, $R=87^\circ$). Render-compare fixes rotation (R drops to 20°). ICP refinement pulls the model close ($T=5\text{mm}$, $R=9^\circ$). Flip correction polishes to $T=9\text{mm}$, $R=2^\circ$ (Excellent tier). The rightmost panel shows the GT target in green.

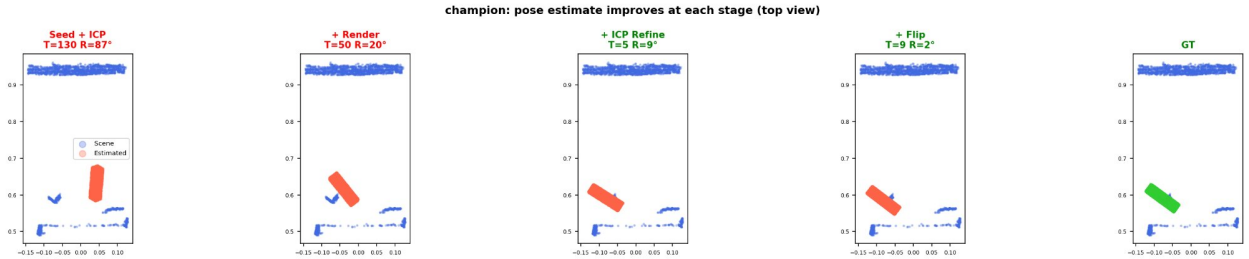


Figure 4: Champion spark plug through four pipeline stages (top view). Teal = scene. Red = estimated pose. Green = GT target. The estimated model moves closer to the scene at each step.

3.2 Multi-View Selection and Result

All 432 images are evaluated independently and the best is selected. Only about 2% of images achieve acceptable accuracy, but 432 per object gives enough candidates. **V1 result: 17/24 (71%).** The 7 failures are all from seeds missing the object: sharpie ($T=55\text{mm}$), safety ($T=80\text{mm}$), dr_browns ($T=29\text{mm}$).

4 V2: PnP + Depth Fusion

V1 fails when seeds miss the object. From Section 2.2, PnP gives $T=3\text{mm}$ on every image regardless of seed quality. So I replaced depth seeds with PnP translation:

1. **PnP Translation:** ORB + depth $\rightarrow$ solvePnPRansac $\rightarrow$ translation.
2. **Render-Compare Rotation:** Same 200-candidate search, anchored at PnP translation.
3. **ICP Refinement:** Polish both T and R.
4. **Smart Flip:** Only flip when confidence < 0.3 and improvement $> 5\%$.

Smart flip was critical. Without it, cheezit config G: ICP gives $R=1^\circ$ (perfect), but naive flip selects $R=179^\circ$ because the flipped silhouette scores slightly higher from noise. Confidence-gating prevents this.

V2 result: 21/24 (88%). Per-image success: $\sim 2\% \rightarrow \sim 5\%$ ($2.8\times$).

5 Results

5.1 3D Pose Overlay and Per-Image Scatter

Figure 5 is the main result. Each row shows one object across four panels: the top-view 3D overlay, the front-view 3D overlay, and the per-image scatter plots for V1 and V2. In the 3D overlays, teal is the observed scene, green is the GT model, red is the V1 estimate, and blue is the V2 estimate. When the estimated model overlaps the GT, the pose is correct.

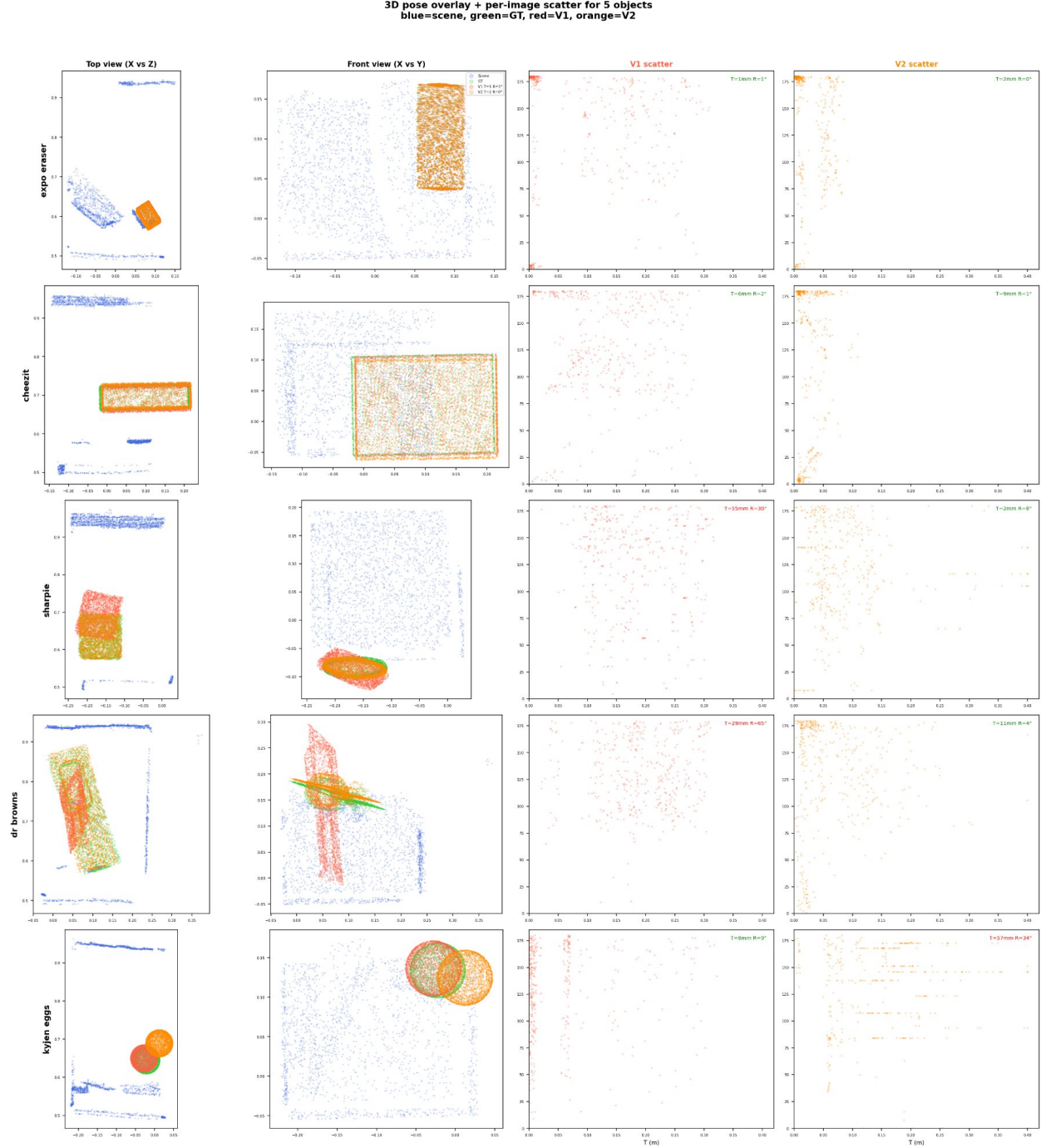


Figure 5: 3D pose overlay + per-image scatter for 5 objects. Teal=scene, green=GT, red=V1, blue=V2. Each scatter dot is one of ~ 432 images.

Key observations from Figure 5:

Expo eraser (row 1): Both V1 red and V2 blue overlap the green GT closely. Scatter shows tight clusters near the origin. Both pipelines work well.

Cheezit (row 2): Both succeed. The highest per-image success rate (16.9%) because the large textured surface provides good features for both depth and RGB.

Sharpie (row 3): V1 red is displaced from green (T=55mm, R=30°). V2 blue overlaps green (T=2mm, R=8°). The V1 scatter is spread everywhere while V2 collapses toward the origin. PnP translation fixed the seed-finding problem for this thin, colourful object.

Dr_browns (row 4): V1 fails (T=29mm, R=65°) because the semi-transparent bottle brush gives unreliable depth seeds. V2 finds it through visible texture (T=11mm, R=4°).

Kyjen eggs (row 5): The opposite pattern. V1 red overlaps green (T=8mm, R=9°) but V2 blue drifts (T=57mm, R=34°). The smooth eggs have no texture features for ORB, so PnP gets unreliable correspondences.

5.2 Complete Results

Table 1: All 24 objects. T in mm, R in degrees. Acceptable: T<25, R<15.

Tier	Object	V1 T	V1 R	V2 T	V2 R	V2 %img	Change
<i>Excellent (T<10, R<5) – 11 objects</i>							
	expo	1	1	2	0	4.9%	
	genuine_joe	3	2	2	0	13.7%	
	crayola	4	1	3	1	8.8%	
	highland	4	2	4	2	6.9%	
	first_years	4	5	4	3	6.2%	
	cheezit	6	2	9	1	16.9%	
	elmers	6	2	7	3	2.1%	
	paper_mate	6	10	2	2	2.6%	
	laugh_book	7	3	1	2	2.1%	
	oreo	7	11	3	2	6.0%	
	champion	9	2	4	2	2.1%	
<i>Acceptable (T<25, R<15) – 10 objects</i>							
	kong_duck	5	8	10	6	1.9%	
	kong_air	14	6	5	8	1.9%	improved
	stanley	10	6	5	7	1.9%	improved
	feline	23	9	5	3	13.6%	improved
	mark_twain	22	8	7	10	2.4%	improved
	kong_frog	14	26	12	12	0.2%	V2 fixed
	mommys	17	21	5	4	15.0%	V2 fixed
	dr_browns	29	65	11	4	3.3%	V2 fixed
	sharpie	55	30	2	8	2.3%	V2 fixed
	safety	80	8	2	6	0.7%	V2 fixed
<i>Mismatch – 3 objects</i>							
	kyjen_eggs	8	9	57	34	0%	regressed
	rolodex	78	30	10	16	0%	close miss
	munchkin	82	21	–	–	–	processing error

5.3 Failure Analysis

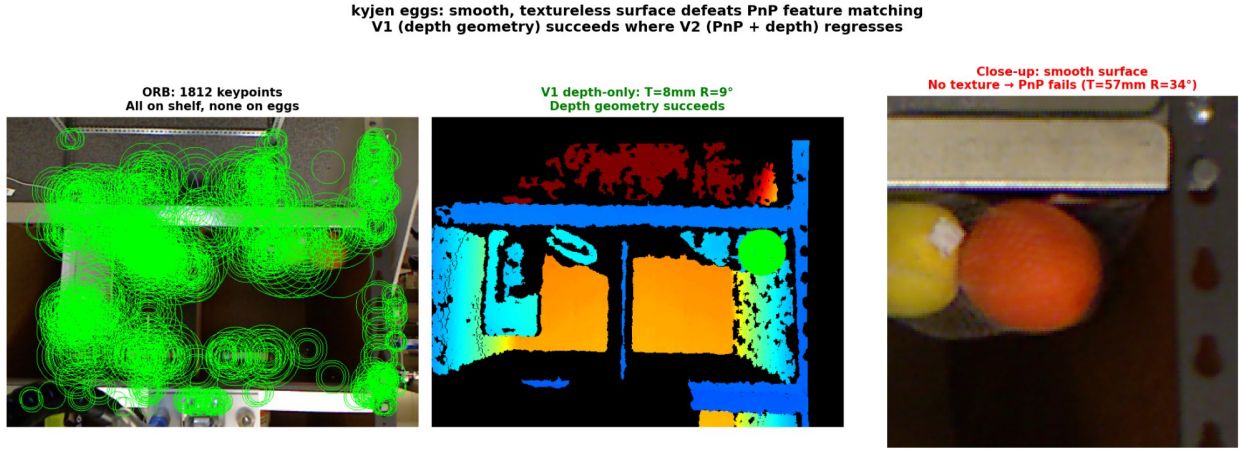


Figure 6: Left: ORB detects 1812 keypoints on the RGB image, but they all land on the shelf structure, not on the smooth eggs. Centre: V1 succeeds ($T=8\text{mm}$, $R=9^\circ$) because depth geometry does not need surface texture. Right: close-up showing the textureless egg surface.

Kyjen eggs (Figure 6): V1 works ($T=8\text{mm}$, $R=9^\circ$) but V2 regresses ($T=57\text{mm}$, $R=34^\circ$). ORB extracts features from the RGB image, but the smooth egg surface provides no distinctive texture. All 1812 keypoints land on the shelf structure instead of the eggs. Without valid correspondences on the object itself, PnP produces an incorrect translation anchor, and the subsequent depth stages cannot recover. This demonstrates that fusion is not universally better.

Rolodex cup: Misses by just 1° ($R=16^\circ$ vs threshold of 15°) due to cylindrical symmetry. The depth silhouette looks the same from many rotation angles around the vertical axis.

Munchkin duck: Both pipelines crash during processing. The object produces inconsistent depth readings, likely due to its reflective surface interacting poorly with the Kinect’s infrared projector.

6 Discussion

6.1 Complementary Modalities

The central finding: depth and RGB are complementary. RGB through PnP gives translation (3mm median) because ORB features provide 2D-to-3D correspondences that pinpoint position. But with sparse features clustered on one face, the EPnP solution [5] cannot constrain rotation (166°). Depth through render-compare gives rotation because silhouettes encode 3D shape. Fusion combines both: PnP anchors translation, render-compare searches rotation, ICP refines.

6.2 Texture Domain Gap

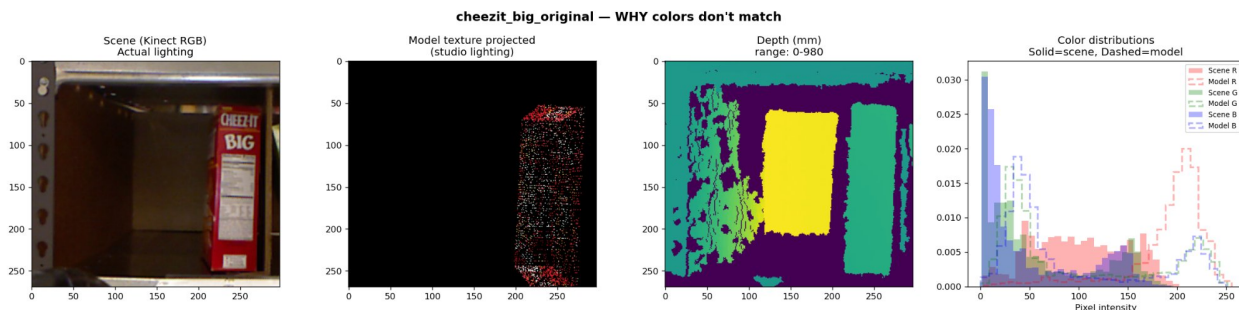


Figure 7: Scene RGB (left, dark Kinect lighting) vs model texture (centre, studio lighting). Colour distributions (right) show a large gap. Illumination correction raises match from 9% to 84% but all candidates improve equally, losing discriminative power.

I explored using model textures (brand text, barcodes) to resolve the 180° flip ambiguity. But the studio-to-Kinect domain gap (Figure 7) makes raw colour matching fail. Illumination correction improved the overall match rate but all rotation candidates benefited equally, so I could not distinguish correct from incorrect orientations. This dead end confirmed depth render-compare as the right rotation approach.

6.3 Related Work

Rennie et al. [1] evaluated LINEMOD [3] on this dataset and found translation-dominated errors from confusing clutter objects with the target. My approach uses 3D geometry directly, avoiding this, but introduces a different bottleneck (seed quality) that V2 fixes. SIFT [7] might give better correspondences than ORB for rotation. A classmate projects GT 3D points to 2D with added noise and recovers with PnP, which bypasses the hard problem of blind feature matching. My approach performs blind matching, which is the realistic scenario. The RGBTrack paper [9] also uses render-and-compare for rotation, confirming this strategy. The BOP benchmark [8] uses symmetry-aware metrics that would count rolodex (1° over) as correct.

6.4 Limitations

The depth prior ($\pm 50\text{mm}$) assumes shelf calibration is available, and the five clustering approaches I tested could not replace it. Processing takes ~ 25 minutes per object due to the exhaustive search across $432 \text{ images} \times 4 \text{ stages}$. Per-image success of 2 to 5% means 432 images are necessary, not excessive. Objects with reflective surfaces (munchkin) cause processing failures because the Kinect depth sensor produces unreliable readings on these materials.

7 Conclusion

Classical methods (ICP, solvePnP, render-and-compare, multi-view selection) achieve industrially relevant pose estimation when fused. RGB gives translation (3mm), depth gives rotation (silhouette matching). V1 17/24 to V2 21/24 validates fusion. Safety glasses improved from T=80mm to T=2mm. The kyjen regression shows fusion is not always better: textureless objects are better served by depth-only geometry. Per-image success of 2 to 5% shows that multi-view data collection is essential for reliable pose estimation.

References

- [1] C. Rennie et al., “A dataset for improved RGBD-based object detection and pose estimation for warehouse pick-and-place,” *IEEE RA-L*, 2016.
- [2] N. Correll et al., “Analysis and observations from the first Amazon Picking Challenge,” *IEEE Trans. Automation Science and Eng.*, 2018.
- [3] S. Hinterstoisser et al., “Model based training, detection and pose estimation of textureless 3D objects in heavily cluttered scenes,” *ACCV*, 2012.
- [4] P. J. Besl and N. D. McKay, “A method for registration of 3-D shapes,” *IEEE TPAMI*, 1992.
- [5] V. Lepetit et al., “EPnP: An accurate $O(n)$ solution to the PnP problem,” *IJCV*, 2009.
- [6] E. Rublee et al., “ORB: An efficient alternative to SIFT or SURF,” *IEEE ICCV*, 2011.
- [7] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *IJCV*, 2004.
- [8] T. Hodáň et al., “BOP: Benchmark for 6D object pose estimation,” *ECCV*, 2018.
- [9] RGBTrack, “Render-and-compare for depth-free 6DOF pose estimation,” 2025.